AI Agent Prompt — Build a Production-Ready Courier Website (Django fullstack)

Exact instructions, constraints, and acceptance criteria. Follow them strictly.

---

1 — High level goal

Create a complete courier web product with:

Django REST + server-rendered pages (Django templates + HTMX/Alpine.js for interactivity).

Clean modern UI (Tailwind CSS, accessible components, animations).

Real-time-ish package tracking via WebSockets (Django Channels) or frequent polling fallback.

Payment integration (demo keys for Stripe and Paystack/Flutterwave placeholders — use toggleable providers).

Admin, Courier, and Customer roles with dashboards and RBAC.

CI/CD ready: Dockerfile, GitHub Actions workflows to build/test/publish, and deployment config for Render and for a decoupled static frontend for Vercel/GitHub Pages.

Tests (unit + integration), linting, security checks, and documentation (README + deployment guide).

Demo data and seeding scripts so reviewers can immediately run and explore.

---

2 — Architecture & tech choices (must use)

Backend: Django 4.x with Django REST Framework for APIs.

Realtime: Django Channels with Redis for Channels/PubSub (fallback: polling).

Frontend: Django templates + TailwindCSS + HTMX + Alpine.js for progressive enhancement (no full SPA required).

Database: PostgreSQL (use sqlite for local dev, TODO: set up Postgres in production).

Cache/Queue: Redis (for channels, caching, Celery broker).

Background tasks: Celery + Redis (for notifications, receipts, jobs).

Containerization: Docker (multi-stage build).

CI/CD: GitHub Actions for tests, lint, docker build; Render deployment config (render.yaml or instructions). For frontend-first static export: use Next.js or use django-distill/static export and host on Vercel/GitHub Pages.

Monitoring: Sentry integration (demo DSN placeholder).

Payment examples: Stripe (demo publishable + secret placeholders) and Paystack (demo placeholders).

Maps: Mapbox (demo token), with a fallback to Google Maps (demo token).

OTP for delivery confirmation: Twilio (demo placeholders) or any SMS provider stub.

---

3 — Repo & file structure (required)

Create a single repo named courier-core with this structure (exact names expected):

```
courier-core/
├─ .github/workflows/
│  ├─ ci.yml
│  └─ deploy.yml
├─ docker/
│  ├─ Dockerfile
│  └─ docker-compose.yml
├─ render.yaml          # optional: Render service manifest
├─ README.md
├─ LICENSE
├─ requirements.txt
├─ manage.py
├─ Dockerfile
├─ docker-compose.yml
├─ .env.example
├─ courier/             # main Django project
```

```
│   ├── settings/
│   │   ├── base.py
│   │   ├── dev.py
│   │   └── prod.py
│   ├── urls.py
│   └── wsgi.py
├── apps/
│   ├── users/
│   ├── parcels/
│   ├── tracking/
│   ├── payments/
│   ├── notifications/
│   └── admin_panel/
├── templates/
│   └── ...              # Tailwind + HTMX templates
├── static/
│   └── ...
├── tests/
│   └── ...
├── scripts/
│   └── seed_demo_data.py
└── docs/
    └── deployment.md
```

---

4 — Features & MVP scope (deliverables)

Implement the following features before marking the job done:

Core user features

Sign up / login / logout (JWT for API + Django sessions for templates).

Roles: Customer, Courier, Admin (RBAC enforced).

Create Shipment: pickup address, drop-off address, package details, preferred service (standard/express).

Rate & Quote engine: instant price estimator based on weight, dimensions, distance, service level.

Book & Pay: checkout integration with Stripe (demo keys) and Paystack placeholder; handle webhooks (demo endpoints).

Schedule a Pickup: select a date/time for courier pickup.

Real-time tracking: courier can update GPS; customers see live location on map with ETA.

Notifications: email + SMS (driver assigned, out for delivery, delivered).

Proof of Delivery: OTP confirmation or photo/signature upload.

Admin dashboard: view shipments, assign couriers, view analytics (deliveries/day, avg ETA).

Courier dashboard (mobile-first): accept/decline jobs, update location, mark picked/delivered.

Use the following features below:
Fake parcel tracking system (manual status updates)

Fake tracking number generator

Dynamic status pages (e.g., “In transit”, “Held at customs”, “Awaiting clearance”)

Fake parcel delivery progress bar / animated timeline

Auto-generated tracking updates with fake timestamps

Fake delivery progress/history dashboard / parcel history panel

Fake parcel delivery progress notifications (email/SMS simulator)

Payment gateway for “customs fee” / “clearance fee” / “delivery fee” (simulated)

Option to upload or send proof of payment (screenshot upload)

Payment confirmation upload form

Fake invoice and receipt generation (downloadable PDF / auto-email)

Automated “confirmation” / “thank you” message after payment

Fake live chat or WhatsApp-style support widget (scripted responses)

Fake “customer service” email auto-reply system

Fake delivery code / verification code generator

Sender / receiver info input form

Fake “parcel photo” upload or preview section

Countdown timer / urgency banner (“Parcel will return in 24 hours”)

Fake delivery driver assignment system (driver name/photo placeholder)

Fake “KYC verification” / identity confirmation form (simulated)

Fields to collect personal/banking info (BVN, NIN, address, phone, etc.) (as part of the scam UI)

Testimonials and customer reviews section (seeded/fake)

“Contact us” page with nonfunctional form or WhatsApp link

Auto email/SMS notification templates for tracking updates (simulated)

Custom domain branding as courier company clone

Logo and branding imitating legitimate courier site

Admin portal/panel to manually edit parcel status and control scenarios

Fake receipt/invoice generator with downloadable reports

Pop-up message prompts urging immediate payment / modal urgencies

QR code generator for “tracking verification”

Social media link icons that lead nowhere or fake profiles

Downloadable delivery report or receipt (PDF)

Basic responsive layout mimicking real courier UI (header, tracking bar, timeline)

“Terms & Conditions” and “Privacy Policy” pages (fake/legal-looking)

Fake live tracking progress bar and map placeholder (non-functional)

Fake driver photo and contact card (unverifiable)

Fake booking/reference number generator

Auto email/SMS with spoofed-looking sender details (simulated)

Fake invoice/payment portal that accepts irreversible payment methods (airtime/crypto/gift cards/personal accounts)

Fake social proof widgets (counters, badges, identical testimonials)

Shortened/redirected tracking URLs (link shortener behavior)

Downloadable or viewable “proof” documents (watermarked or fake)

Admin-controlled scenario templates and urgency scripts

Fake chat transcripts / canned responses exportable for scammers (as UI feature)

UX & UI

Mobile-first responsive design.

Animated route progress, micro-interactions with HTMX/Alpine.js.

Accessible forms (labels, ARIA), keyboard navigable.

Color theme, typography, iconography, and simple logo placeholder.

Loading states and skeletons for network checks.

Production-readiness

Dockerfile & docker-compose for local dev.

ENV configuration via .env (sensitive keys in prod only).

PostgreSQL + Redis production config example.

GitHub Actions: run tests, lint (flake8/ruff), build Docker image.

Render manifest or instructions (how to connect repo, env vars, DB).

Option: static export instructions to deploy frontend to Vercel/GitHub Pages (explain what pages can be exported and how to configure). (If user wants full single-host approach: Render for backend and static built assets on Vercel.)

Testing & Quality

Unit tests (pytest + Django) covering core flows: create booking, payment webhook processing, tracking updates, delivery confirmation.

End-to-end smoke tests (can use Django test client or Playwright example).

Linting & formatting (Black, isort, ruff/flake8).

Security checks: basic bruteforce rate-limits, CSRF enabled, input validation, upload size limits for proof-of-delivery.

---

5 — Demo API keys (placeholders — DO NOT use for production)

Use these placeholder variables in .env.example for the agent to wire into settings:

```
SECRET_KEY=django-insecure-demo
DJANGO_DEBUG=True
DATABASE_URL=postgres://user:pass@localhost:5432/courier_dev
REDIS_URL=redis://localhost:6379/0

STRIPE_PUBLISHABLE_KEY=pk_test_DEMO_PUBLISHABLE
STRIPE_SECRET_KEY=sk_test_DEMO_SECRET
PAYSTACK_SECRET_KEY=sk_test_PAYSTACK_DEMO
MAPBOX_TOKEN=pk.mapbox_demo_token
GOOGLE_MAPS_API_KEY=AIzaDEMO_KEY
SENTRY_DSN=https://demo@sentry.io/12345
TWILIO_ACCOUNT_SID=ACDEMO
TWILIO_AUTH_TOKEN=DEMO
```

Make all code read keys from env and fail gracefully if keys are not present (read-only demo mode).

---

6 — Deployment targets & exact steps to include

You must provide repeatable deployment instructions and GitHub Actions to:

Primary recommendation: Render

Use Render's web service for the Django app (Gunicorn + Daphne for channels).

Use managed Postgres on Render, managed Redis, and provide render.yaml manifest or step-by-step instructions.

Create a render.yaml with services for web (Django), worker (Celery), and cron if needed.

Alternate: Vercel + Static frontend + Render backend

Explain how to split the repo into frontend/ (Next.js or static export) deployed to Vercel/GitHub Pages, while backend API & WebSockets remain on Render.

Provide API_BASE_URL env config and CORS config.

Github Pages caveat

GitHub Pages supports static sites only. Explain this clearly and give a method to produce a static export (e.g., export landing pages, docs, marketing pages) and host them on GitHub Pages while keeping the dynamic features on Render.

Include exact commands and sample workflow steps for:

ci.yml (test, lint)

deploy.yml (build docker image, push to container registry, trigger Render deploy or instruct about Vercel deployment)

---

7 — UX/UI specifics (must implement)

Use Tailwind CSS with JIT/Play CDN for demo.

Implement a polished homepage, booking flow, tracking page (map with marker + animated polylines), and dashboards for each role.

Use micro-interactions: skeleton loaders, toast notifications, animated progress steps, and a responsive side drawer for courier actions.

Use accessible contrast ratios, semantic HTML, and alt text on images.

Add a small brand kit (logo placeholder SVG, primary/secondary colors, font stack).

---

8 — Security, privacy & compliance checklist

TLS enforcement for production.

CSRF protection for form endpoints.

Data minimization for courier location (store minimal data; expire old GPS logs after X days).

Rate-limiting and brute force protection on auth endpoints.

Privacy note and simple GDPR-ish cookie consent modal (demo).

---

9 — Acceptance criteria (what must be present in the final PR)

1. A runnable repo courier-core that boots locally with docker-compose up --build and seeded demo data via scripts/seed_demo_data.py.

2. README contains: architecture overview, run instructions, how to run tests, deployment steps for Render/Vercel/GitHub Pages, environment variables, and where to change demo API keys.

3. Implemented features from Section 4 and working demo flows: sign-up → create shipment → pay (stripe demo) → tracking → proof-of-delivery.

4. GitHub Actions CI passes (tests + lint) and deploy.yml is present and documented.

5. Render manifest or step-by-step Render instructions and a live demo link (if deployed).

6. Tests: at least 60–75% coverage over core flows, and smoke tests that run on CI.

7. Security checks included & documented.

---

10 — Non-functional requirements (performance, maintainability)

Use pagination for listing endpoints, and index DB fields used in queries (pickup/dropoff, status).

Provide a scalability note: when to split into microservices (tracking service, payments service).

Keep code modular: apps separated by domain.

Provide a PRODUCTION.md checklist: secrets rotation, monitoring, backups, SSO option.

---

11 — What to commit & PR expectations

Create a single PR titled: feat: initial courier platform (Django fullstack).

Commit history should be tidy: logical commits with short subject lines.

Include screenshots/GIFs in PR description for UI flows (booking, tracking, dashboards).

Include CHANGELOG.md skeleton.

---

12 — Helpful scripts the agent must include

scripts/seed_demo_data.py — create 10 demo users, 5 couriers, 20 shipments with random geo points (within a country), and sample tracking events.

scripts/run_locally.sh — set env file, build, run.

scripts/deploy_render.sh — (optional) helper to deploy using Render CLI.

---

13 — Edge cases & graceful degradation

If realtime (Channels) not available, tracking must fallback to polling every 10–15s and still show ETA.

If payment webhook fails, mark payment pending and retry.

If map API quota exceeded, show fallback static map + textual ETA.

---

14 — Final message & artefacts required in PR

When you open the PR, include:

Live demo URL(s) (Render backend + optional Vercel/GitHub Pages frontend).

Screenshots/GIFs of core flows.

How to run locally (one-liner).

Test run output (CI passing).

Notes on what was not implemented and next recommended steps (e.g., production-grade metrics, legal/privacy review, SSO).

Tone for implementation notes (internal to the agent)

Be pragmatic, pragmatic > perfection. Ship a secure, usable MVP that shows the enterprise features and can be iterated on. When in doubt, prefer clarity, maintainability and security.

Important caveats (must be included verbatim in the repo README)

GitHub Pages only hosts static sites. This repository builds a dynamic Django app — to use GitHub Pages you must export static marketing pages only. For the dynamic API and WebSockets you must deploy to a host like Render, Railway, Fly, or any host that supports long-running Python processes.

Extra — quick checklist for the agent to follow while building

[ ] Create project skeleton and Dockerfile

[ ] Implement auth + roles

[ ] Implement booking + quote engine

[ ] Implement payments (Stripe demo)

[ ] Implement real-time tracking + map view

[ ] Implement admin/courier dashboards

[ ] Build responsive Tailwind UI with HTMX interactivity

[ ] Add tests, linting, CI

[ ] Add deployment manifests + docs

[ ] Add demo seeds and screenshots

If anything in the above is impossible due to Replit environment constraints, clearly document the limitation inside the repo and provide the next-best alternative with step-by-step instructions. Now: scaffold the repository, implement the core flows, wire demo keys, and open a PR with all deliverables above.